PORTFOLIO · 2026

Selected work

Shipped cross-platform apps, a live consumer product, real-time and location-aware systems, and applied AI. Each public project below can be shown running, from a demo path in its repository.

01	NoNonsenseMusic: native Mac, Android and Windows apps with a self-hosted server	shipped, v0.3.1
02	Arya: multi-camera face recognition for unique-visitor counting	computer vision
03	Citizen complaints platform: OTP, GPS, evidence uploads, officer back office	backend
04	SonicNet: SOS relay over a phone-to-phone mesh, with GPS and triage	real-time, location
05	Production AI work: document extraction, agentic ETL, learning agents	LLM systems
06	Unvelit: a live anonymous college forum, and the moderation behind it	live, unvelit.in
07	BrightWords: AI English tutor for children, with a parent view	web app + LLM
08	Conversational AI: lead capture over web, WhatsApp and voice	RAG, agents
09	Reveal, and smaller builds	

Sample people in demos are synthetic (GAN-generated faces) or fictional. Screens are from running builds.

FOR A HEALTHCARE ON-DEMAND APP

Where each part has been built before

A Practo- or Uber-style healthcare product is a set of familiar parts: patient and provider apps, verified accounts, location, live status, an operations console, documents, and AI on top. This is where each part already exists in the work that follows.

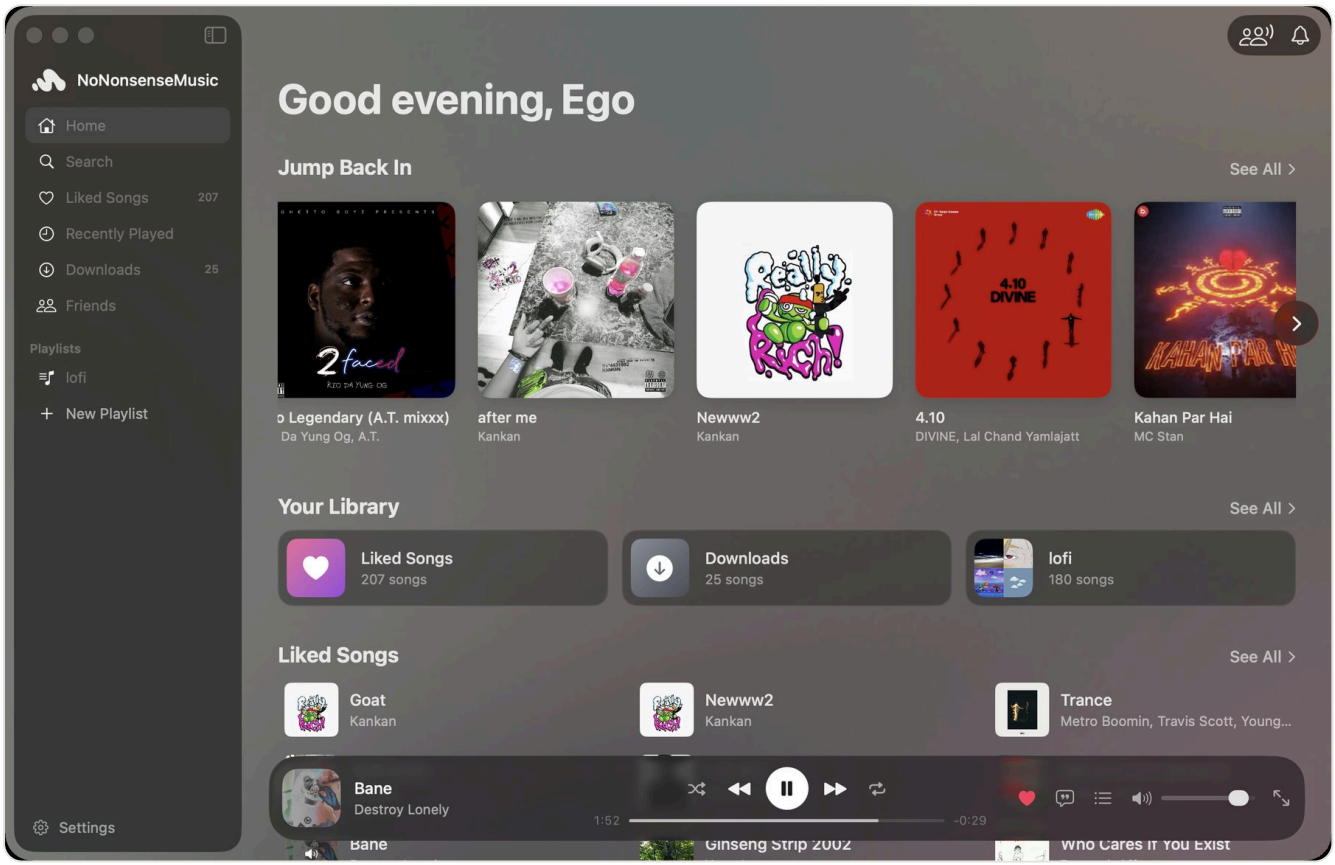
CAPABILITY	BUILT IN
Patient and provider apps on phone and desktop	**NoNonsenseMusic**: a native SwiftUI Mac app and a Flutter app for Android and Windows from one codebase, released, with offline downloads, background playback, lock-screen controls and a home-screen widget.
Verified sign-up, sessions, roles	**Complaints platform**: phone OTP with attempt limits and single-use action tokens, JWT access and rotating refresh tokens, citizen and officer roles. **NoNonsenseMusic**: multi-user accounts with recovery codes.
Location on every request	**Complaints platform**: GPS stored on logins, complaints and each evidence file. **SonicNet**: GPS fix and place description on each SOS, carried hop to hop.
Live status and real-time rooms	**NoNonsenseMusic**: Jams, where several devices share one queue and play in sync, with the server as the source of truth. **SonicNet**: a command centre that receives SOS calls live over WebSocket.
Dispatch and triage console	**SonicNet**: urgency from type and message, acknowledge flow, route of each call. **Complaints platform**: officer dashboard, pending queue, status changes with an audit log.
Reports, prescriptions, photos	**Complaints platform**: image and video evidence, type and size checks, compression, per-complaint storage. **Unvelit** (live at unvelit.in): automated screening of every upload.
Notifications and follow-up	**NoNonsenseMusic**: friend requests, shares and invites as live notifications. **Lead platforms**: email and WhatsApp follow-up sequences that pause when the person replies.
AI assistants and triage	**Lead platforms**: RAG chatbots on web and WhatsApp with human handoff. **Production AI**: source-grounded extraction from long documents. **BrightWords**: spoken-answer scoring.
Identity and computer vision	**Arya**: face detection, embeddings and matching across cameras, sized for a 300,000-person event.
Security and privacy	Rate limiting, input sanitisation, security headers and audit trails (complaints platform); no telemetry and no IP or search logging (NoNonsenseMusic).

01 · SHIPPED PRODUCT

NoNonsenseMusic

A music player with native apps for macOS, Android and Windows and a server that a group of friends can share. Open source, released (v0.3.1), and in daily use by its author.

Swift 6 · SwiftUI | Flutter 3 · Dart | Python 3.13 · FastAPI | PostgreSQL | WebSockets | AGPL-3.0



Mac app, home. The window takes its colours from the current cover; ~0.3% CPU when idle.

3	55k	576	8
platforms from two client codebases	lines across Mac, Flutter, server and tests	server tests passing	public releases

What it does

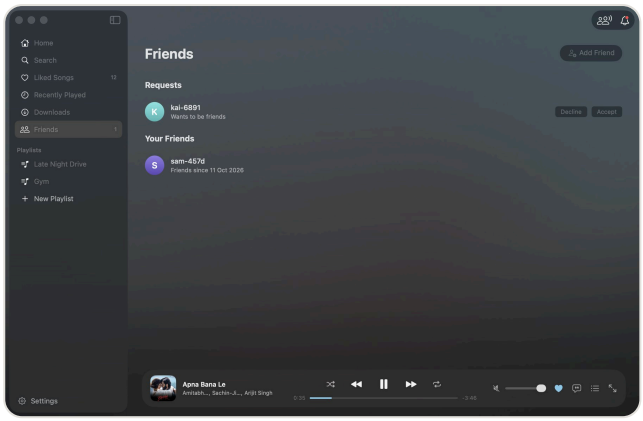
- One search across JioSaavn, YouTube Music and YouTube; the same recording from several sources is shown once and the best copy plays.
- Shared and public playlists, friends, and notifications.
- **Jams**: friends listen together on their own devices, in step, from one shared queue.
- Timed lyrics with line-by-line translation, on-device on the Mac.
- Offline downloads, prefetching of the next song, autoplay.

Engineering

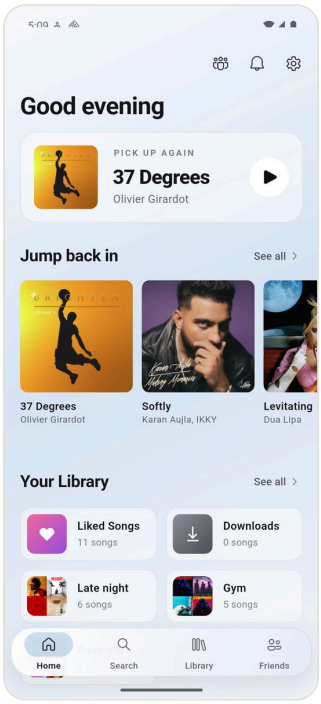
- Jam rules are written once (`docs/jam.md`) and implemented in all three codebases, with the server as the only source of truth and a shared playhead model.
- Rooms survive server restarts: written on shutdown, resumed if back within 60 s.
- Numbered SQL migrations; argon2 passwords; per-user rate limits behind a trusted proxy.
- Privacy by design: no telemetry, no IP or search logs; YouTube lookups run on the user's device.

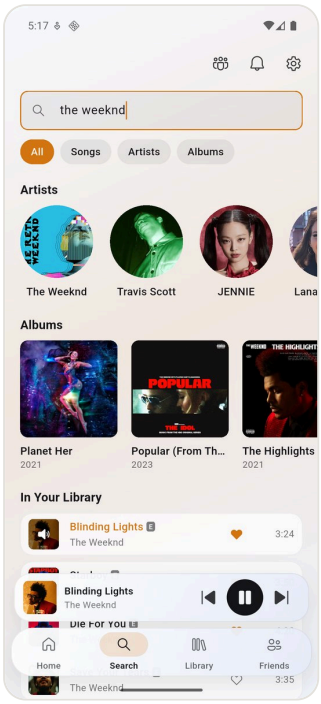


Timed lyrics with translation (Punjabi to English).

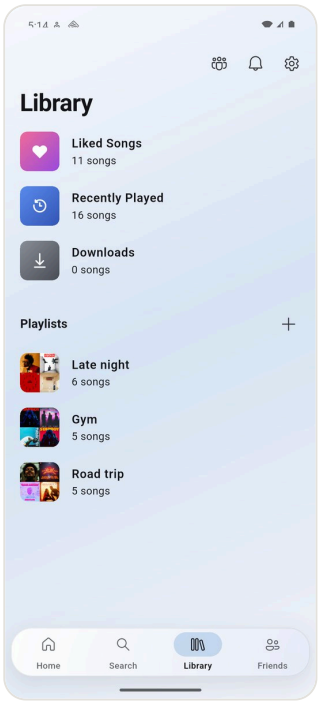


Friends, shared playlists and invitations.









Android app (Flutter): home, search, now playing, library. Background playback with lock-screen controls and a home-screen widget.

DEMO PATH

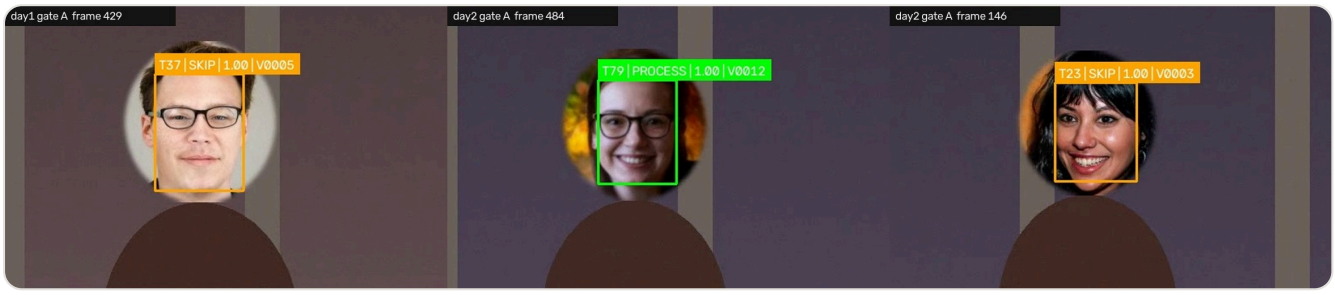
Install the Mac, Windows or Android build from the latest GitHub release, point it at a server (`uv run fastapi run src/music_backend/main.py` with PostgreSQL), sign up two accounts and start a Jam: both devices play the same queue in step. Built with two contributors, credited in the README (Windows app and phone redesign).

02 · COMPUTER VISION

Arya: unique-visitor counting

Counts how many different people passed through an event from the footage of several cameras over several days. Built for a three-day event with more than 300,000 expected attendees.

PyTorch · MTCNN · FaceNet (InceptionResnetV1, VGGFace2) · ChromaDB · HNSW · cosine · OpenCV



Annotated frames from the demo run. Green: face embedded and matched against the store. Orange: same person already tracked, so FaceNet is skipped. Labels: track, decision, detection confidence, visitor ID.

14 / 14
people counted exactly once across 3 cameras and 2 days (20 appearances)

0
people counted twice or merged with someone else

57%
of face detections skipped by the tracker, never sent to FaceNet

Pipeline

- MTCNN finds faces; small or low-confidence detections are dropped.
- A light IOU tracker follows each face within a clip, so a person walking past is embedded once, or again only when a sharper view appears.
- FaceNet turns each face into a 512-d embedding; ChromaDB (HNSW, cosine) finds the nearest known visitor across all cameras and days.
- Known visitors keep a quality-weighted running average of their embeddings, which gets more robust with each sighting.

Built for a long, large run

- Checkpoints every 100 processed frames; a stopped run resumes where it left off.
- Persistent vector store, so later days match against earlier ones.
- Exports per-visitor records (first seen, cameras) to CSV and JSON.
- Batch embedding on GPU; frame sampling configurable per deployment.

DEMO PATH

`demo/run_demo.sh` renders three camera clips of synthetic people walking past (with overlap between gates and returning visitors), runs the pipeline and checks the count against the known answer. Drop real footage in `videos/` and run `python main.py` for the real thing.

03 · CITIZEN-FACING PLATFORM

Citizen complaints platform

The backend for a police complaints app: citizens verify their phone, file complaints with location and evidence, and track them; officers triage them from an admin side. The same shape as a patient app with a clinic back office.

FastAPI | Async SQLAlchemy | MySQL | JWT | Pillow | OpenAPI

Citizen side

- Phone OTP: 5-minute codes, 3 attempts, then a single-use action token to register or reset a password.
- Login records GPS position, IP and device in a login log.
- Complaints by type and category (validated pairs), free text, GPS, photos and video.
- Evidence pipeline: type and size checks, EXIF orientation, resize and recompression, dated per-complaint storage.

Officer side

- Dashboard by status and category with resolution rate; pending queue; user blocking; MIS report.
- Every status change and note goes to an audit log.

Hardening

- Rate limits on OTP, login and complaints; XSS, SQL and path-traversal checks; security headers; request audit logging.
- Access tokens last 15 minutes; refresh tokens are hashed at rest and rotated on use.

```
$ python scripts/demo_walkthrough.py
▶ Citizen asks for an OTP
  HTTP 200 {"message": "OTP sent to
98******56", "expires_in": 300}
▶ A wrong OTP is rejected
  HTTP 400 {"detail": "Invalid OTP. 2
attempts remaining."}
▶ Right OTP → single-use action token
  HTTP 200 {"action_token": "DqE8Xtbu…",
"expires_in": 300}
▶ Register, then log in from GPS
  HTTP 200 {"status": "success", "user":
{"name": "Priya Raman", …}}
▶ Mismatched type and category
  HTTP 400 {"detail": "Category
'burglary' is not valid for complaint
type 'cyber'"}
▶ Burglary complaint + geotagged photo
  HTTP 200 {"complaint_number": "CMP-
2026-563983",
            "latitude": "12.97598",
"longitude": "80.22121",
            "media_count": {"images": 1}}
  → script tag stripped before storage
▶ Officer: dashboard, queue, accept with
note
  HTTP 200 {"status": "accepted",
"admin_notes":
            "FIR registered. SI Karthik
assigned; site visit today 5 pm."}
▶ Citizen sees the update
▶ Someone else's complaint
  HTTP 404 {"detail": "Complaint not
found"}
▶ OTP flood from one client
  status codes: [200, 200, 200, 200, 429,
429, 429]
```

DEMO PATH

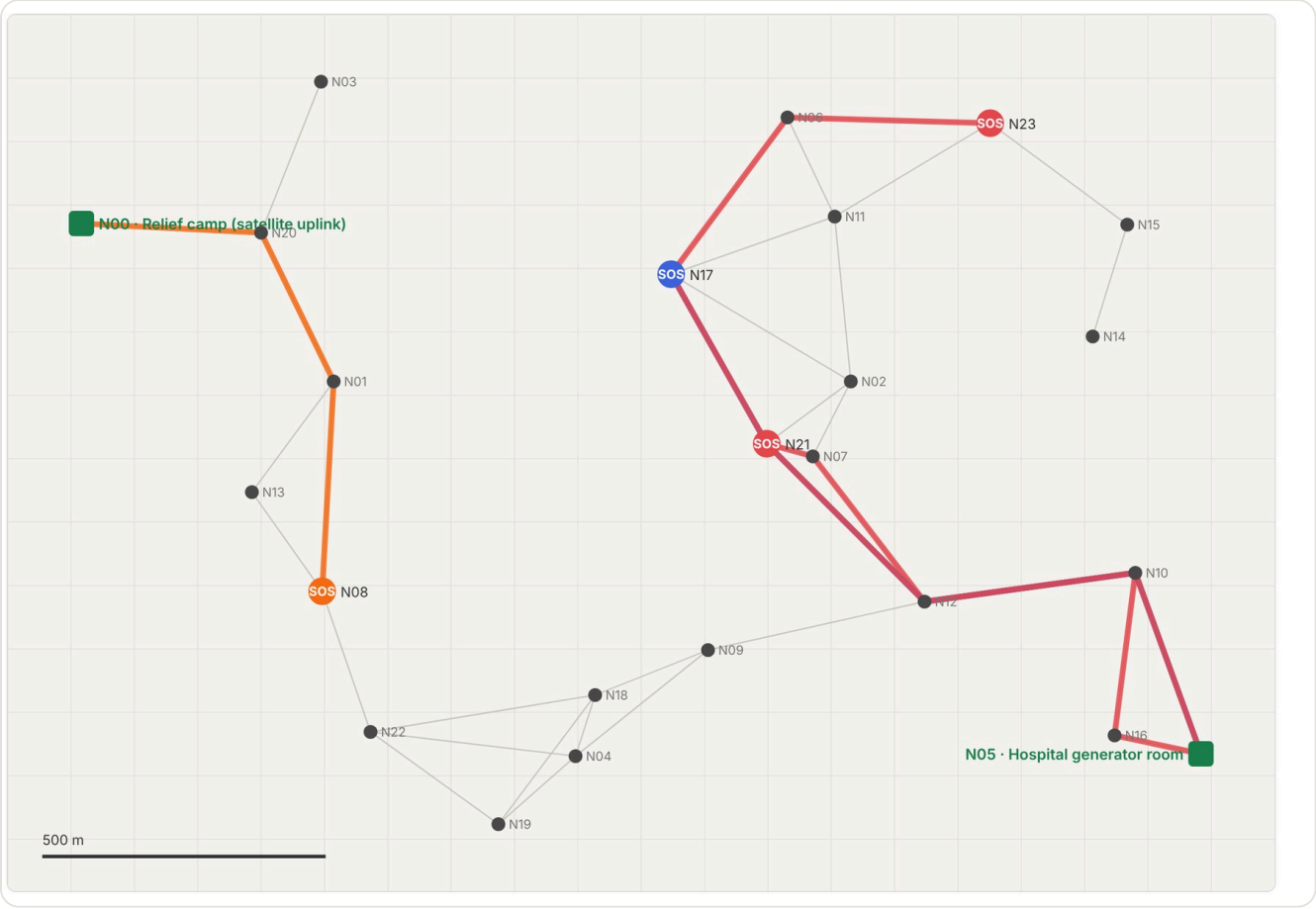
MySQL, `python scripts/init_db.py`, `python -m scripts.seed_admin`, `uvicorn app.main:app`, then `python scripts/demo_walkthrough.py` (output above). Every endpoint can be tried by hand at `/policeapp/docs`.

04 · REAL-TIME, LOCATION, OFFLINE

SonicNet: SOS without a network

Gets an emergency call (type, message, GPS fix, place) from a phone with no signal to a rescue command centre, by relaying it across nearby devices over Bluetooth LE discovery, Wi-Fi multicast or sound (data-over-audio), until one has an uplink.

Python · UDP multicast · GGWave audio modem · BLE (bleak) · Flask-SocketIO · SQLite · HMAC



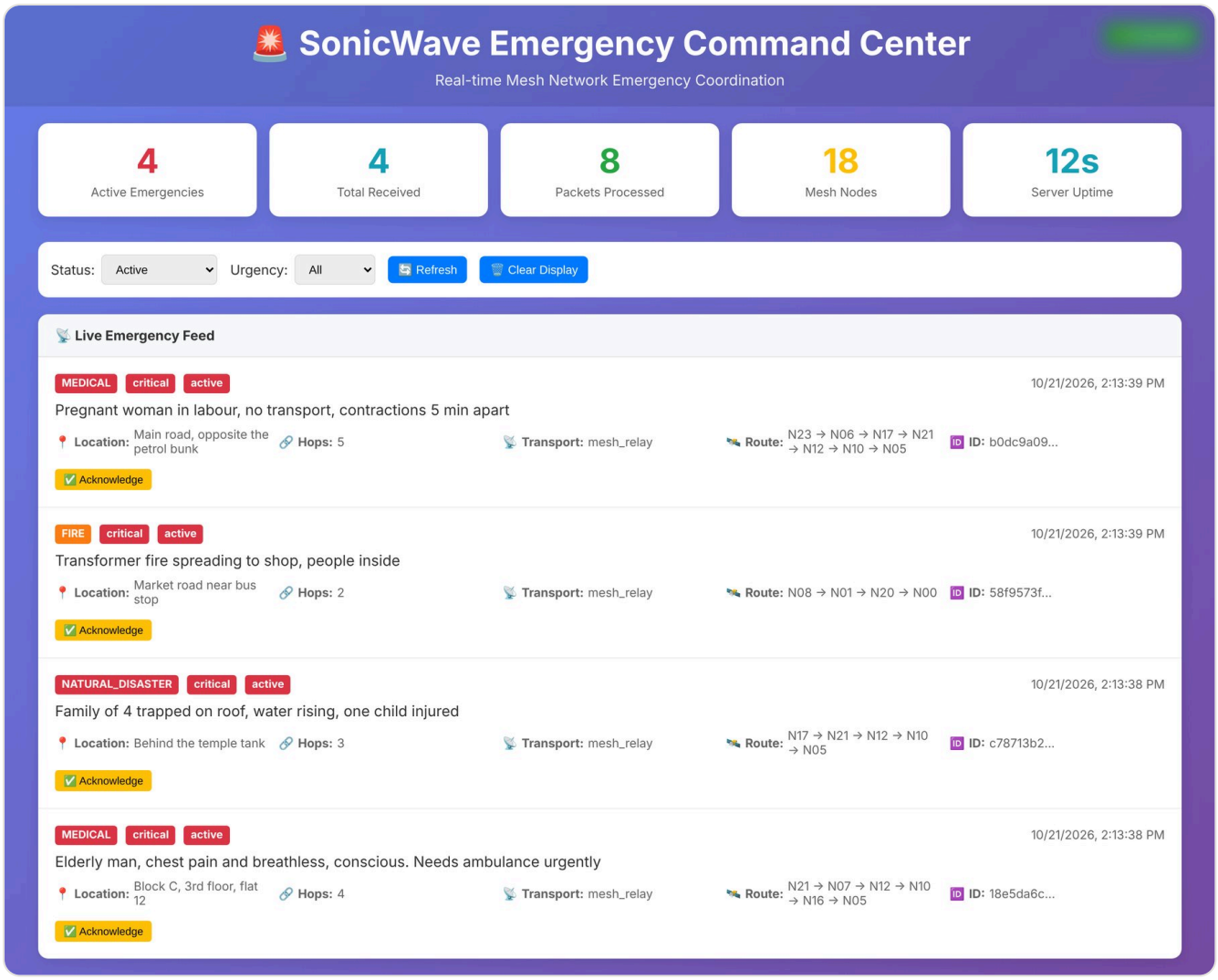
One run of the mesh demo: 24 devices, two with an uplink (green). Each coloured line is the route an SOS actually took to the server.

Relay protocol

- Every packet has a SHA-256 identity hash. A device forwards an SOS at most once: in the run above each call was heard about 60 times but repeats were dropped (149 of 243).
- Hop count with a cap, so packets cannot circulate.
- Server first, relay second: a device with an uplink delivers and stops the flood.
- The server stores a call once even when it arrives through several gateways.

Command centre

- Calls arrive live over WebSocket, triaged by type and message keywords (medical, fire, disaster → critical).
- Each call shows its location, relay count and route, and can be acknowledged.
- Audio path: an HMAC-signed, chunked rescuer ↔ survivor status exchange over speaker and microphone, for devices with no radio at all.



The command centre after the run: a woman in labour, a family trapped by a flood, a transformer fire and a cardiac case, each with its route through the mesh.

DEMO PATH

`python emergency_server.py`, then `python demo/mesh_sim.py`. The demo runs the project's real client (packet hashing, de-duplication, hop limit, relay decisions) on 24 simulated devices; only the radio is simulated, with 420 m range, jitter and 8% loss. The UDP and audio transports have their own laptop-to-laptop test scripts. Status: a working prototype, not yet a phone app; BLE is used for discovery, not yet for relaying.

05 · PRODUCTION AI (PRIVATE REPOSITORIES)

LLM systems in production

Most day-to-day work is in private client repositories, so there are no screenshots here. The architecture of each can be walked through in detail.

Tender document extraction engine

Turns tender bundles of 40+ pages into structured requirement data that can be checked against the source.

- **Verbatim-quote grounding**: every extracted requirement carries the exact text it came from, so nothing is invented.
- **Amendment chains**: later corrigenda are reconciled against the original clauses.
- **~10× lower inference cost** than the first version.

LLMs · structured extraction · evaluation

Self-correcting agentic ETL (US startup)

An ETL engine in which LLM agents transform inconsistent source files into one schema, check their own output and correct it before it is loaded.

LLM agents · data pipelines

Agent pipelines for a learning platform

Agent pipelines behind an online learning platform.

LLM agents

Serenity (research)

A long-running agent runtime: governed input, an append-only SQLite event journal with deterministic replay, bounded working memory, and live model calls whose outputs are validated before they may change state. About 88,000 lines of Python with 64 test modules.

event sourcing · agent architecture · testing

06 · LIVE PRODUCT · UNVELIT.IN

Unvelit: an anonymous college forum

Unvelit (**unvelit.in**) is an anonymous forum for college students, built and run by our team. Anonymous posting with image and video uploads is a risky combination, so every post passes through our own moderation service before anyone sees it. Paid moderation costs money per call, so a repost of something already moderated is recognised and its verdict reused.

FastAPI · MySQL · perceptual hashing · SightEngine · Gemini · OpenCV

How it works

- Images: a 64-bit perceptual hash per upload. Near-duplicate lookup is a single MySQL query, `BIT_COUNT(hash ^ ?) ≤ 8`: Hamming distance as a popcount, with no vector database.
- Video: five sampled frames hashed and screened one by one; screening stops at the first frame that fails.
- Text: Gemini moderation with conversation context.
- Batch endpoints run concurrently; every decision and its reason go to an audit table.

Found and fixed while building the demo

The first version used 16-bit hashes, where about 60% of unrelated images look alike, so a new upload could inherit some other image's "pass". Moving to 64-bit hashes took wrong matches from 4 in 9 to 0 in 9 with the same repost detection, and old rows migrate automatically.

```
$ python demo/dedup_demo.py
Stored as already moderated: 10 images
(3 flagged, 7 passed)

Reposts (should reuse the original verdict):
  recompressed (JPEG q=35)  10/10  dist 0-2/64
  downscaled to 40%         10/10  dist 0-2/64
  brightened +25%           10/10  dist 0-6/64
  cropped 6% off each side   1/10
  watermark strip added      3/10
  34/50 caught: 34 moderation calls saved

Never-seen images (should not match):
  9/9 no match, nearest 16-24/64 bits away

Previous scheme (16-bit):
  new images wrongly matched: 4/9
Current scheme:
  new images wrongly matched: 0/9
```

Perceptual hashes are built for re-encodes and resizes; crops need local features or embeddings, a natural next step.

DEMO PATH

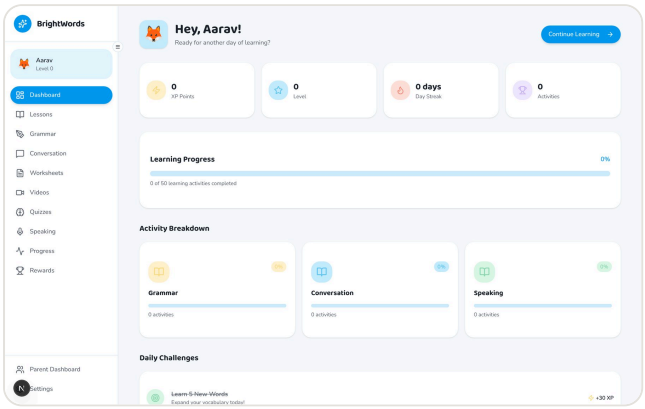
The product: **unvelit.in**. The moderation engine on its own: MySQL, then `python demo/dedup_demo.py`, with no API keys needed (output above). With SightEngine and Gemini keys, `uvicorn api:app` serves the full moderation API.

07 · WEB APP WITH AN LLM

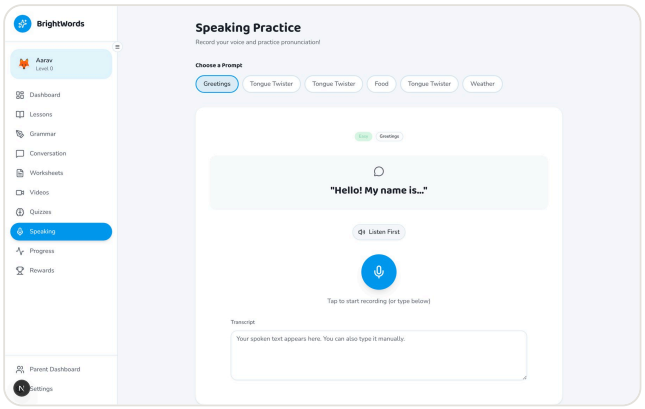
BrightWords: English practice for children

An English-learning web app for ages 6 to 21: conversation practice with an AI tutor, spoken answers scored on fluency, pronunciation, grammar and vocabulary, grammar correction, lessons and quizzes, and a summary view for parents and teachers.

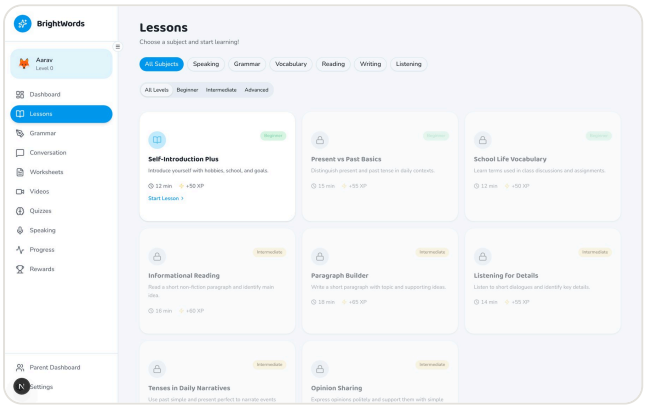
Next.js 16 · React 19 | Tailwind · Radix | FastAPI | LangChain · Ollama | Postgres / SQLite | JWT



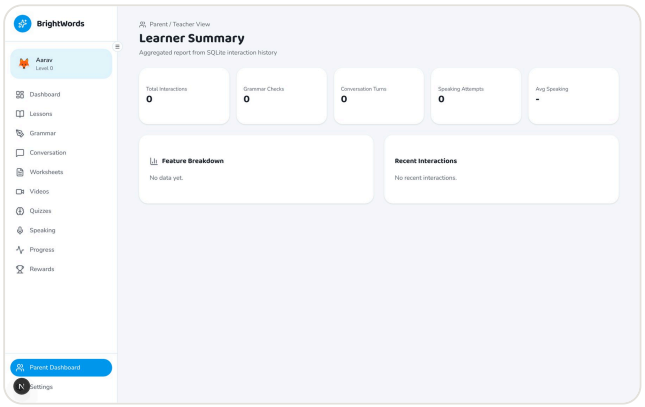
Learner dashboard.



Speaking practice: browser speech recognition, then an AI rubric score.



Lessons by subject and level; difficulty presets by age band.



Parent and teacher summary, aggregated from the interaction history.

DEMO PATH

`uvicorn app.main:app` in `backend/`, `npm run dev` at the root, sign up, open Speaking. AI replies need an Ollama endpoint in `backend/.env`; without one the backend returns safe fallbacks so the rest of the app still works.

08 · CONVERSATIONAL AI

Lead capture over web, WhatsApp and voice

Three iterations of the same business problem, an assistant that answers from a company's own content and turns conversations into qualified leads, from a managed multi-client service to a single agent graph.

LeadGenius

A managed chatbot service for many client businesses. An admin onboards a client and crawls their site into a vector store; the widget answers with RAG and detects leads through LLM function calling.

Celery workers run email and WhatsApp follow-up sequences that pause when the lead replies; typing *TALK* hands the chat to a human. Client dashboards show leads and chat history.

FastAPI · PostgreSQL · Celery · Redis · Supabase pgvector · Gemini

All-in-One lead platform

One knowledge base behind a website chatbot, a WhatsApp Business bot (Twilio) and a browser voice agent built on CrewAI, with all leads and transcripts in one dashboard.

Built from a full requirements specification and design document, with tests for the API, models, services and webhooks.

FastAPI · LangChain · CrewAI · Twilio · PostgreSQL

AutoStream agent

A LangGraph state machine for a SaaS sales funnel: inquiry, qualification, capture. RAG over pricing and features, slot filling into a typed state, structured output for the captured lead, and session memory through a checkpointer.

Chosen over a multi-agent framework because a sales funnel needs deterministic transitions. Ships with Docker.

LangGraph · Gemini 2.5 Flash · Pydantic · Docker

Real-time voice bot pipeline

A WebRTC media server in Node.js with streaming voice-activity detection and barge-in: when the caller starts speaking, the bot's reply is cut off, with a 300 ms target. Raw 16-bit, 16 kHz PCM with a 10 ms heartbeat that keeps the RTP stream alive during silence. Speech recognition and synthesis are stubs, ready for any STT/TTS provider.

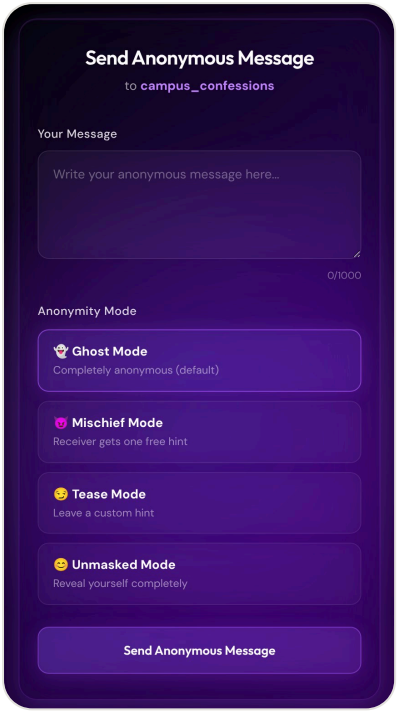
Node.js · WebRTC (wrtc) · WebSocket · state machine

Data analysis agent

Upload a CSV and ask in plain language; the agent writes pandas and Plotly code, runs it in a restricted sandbox (allow-listed imports, time limits) and returns interactive charts, statistics and insights.

FastAPI · LangChain · Gemini · pandas · SciPy · Plotly

Also built



Reveal, the public message page (phone width).

Reveal: anonymous messaging with a hint game

Anyone with a user's link can send them an anonymous message. The sender picks how anonymous to be: Ghost (fully anonymous), Mischief (the receiver gets one free hint), Tease (the sender leaves a custom hint) or Unmasked.

The receiver unlocks tiered hints about who sent it (level 1 to 3), built from browser data captured with the message, and earns hint credits by watching ads. The inbox supports starring, archiving and reporting; messages are moderated and rate-limited.

FastAPI · Supabase (Postgres) · JWT · Jinja · rate limiting

SkillSwap (hackathon)

Matches people who can teach a skill with people who want to learn it: Sentence-BERT profile embeddings, skill overlap and hour-by-hour availability overlap combined into a score with a plain-language explanation; credit transactions between users.

FastAPI · Sentence-BERT · React · PostgreSQL

Emotion classification (GoEmotions)

Multi-label classification of 27 emotions over 58,000 Reddit comments with a BiLSTM and self-attention.

PyTorch · NLP

StreamPlay

A plug-and-play "now playing" overlay for OBS, XSplit and Streamlabs, driven by Windows Media Player.

Batch · VBScript

Stack

Python, TypeScript, Swift, Dart, C/C++, SQL · FastAPI, Flask, Next.js, React, SwiftUI, Flutter · PostgreSQL, MySQL, SQLite, Supabase, Redis, Celery · PyTorch, LangChain, LangGraph, CrewAI, RAG, ChromaDB, pgvector · WebSockets, WebRTC · Docker.